

Lab 02 · El proveedor

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

60 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Qué es Spring, en una frase que se puede comprobar: tú declaras lo que necesitas y otro decide quién te lo da. Se demuestra rompiéndolo — con dos candidatos posibles, la aplicación se niega a arrancar.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. El proveedor, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · El dato y el contrato	3
3.2. Paso 2 · Una implementación, y una ventanilla que la usa	5
3.3. Paso 3 · La pregunta del laboratorio	7
3.4. Paso 4 · Dos candidatos: la aplicación deja de arrancar	8
3.5. Paso 5 · Las dos formas de resolverlo	10
3.6. Paso 6 · La capa del medio	11
4. Lo que aprendiste	14
5. Para profundizar	15
6. Antes de cerrar	15

1 Antes de empezar

1.1 Qué vas a lograr

Este es **el laboratorio que explica qué es Spring**. Los demás enseñan a usarlo; éste enseña qué hace.

Vas a escribir un controlador que no construye nada: declara lo que necesita y lo recibe. Vas a tener dos implementaciones posibles de lo mismo, vas a ver la aplicación **negarse a arrancar** porque no sabe cuál darte, y vas a resolverlo de las dos formas que existen. Al final vas a tener un endpoint que te dice, en tiempo de ejecución, **quién te está atendiendo de verdad**.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 00 y 01 hechos	Sabes arrancar y llamar a un endpoint	—
Estar en la carpeta del lab	<code>cd labs/lab-02-di/practica</code>	El cd no da error
Dos terminales	Una corriendo la app, otra para los curl	—

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de cada línea, y a veces una línea larga se parte en dos. Con Java casi nunca importa; si una línea se parte, el editor te la

marca y basta con unir las. El código completo está en `labs/lab-02-di/solucion/` por si algo se pega mal.

1.4 La puesta a punto

```
cd labs/lab-02-di/practica
```

Esta aplicación escucha en el puerto **8083**. Se para con `Ctrl+C` y hay que reiniciarla después de cada cambio.

2 El caso

La oficina de la DGT tiene una ventanilla que atiende consultas de productos. La oficina **no fabrica nada**: pide lo que necesita a un proveedor.

2.1 El proveedor, que es la metáfora de este laboratorio

La ventanilla no fabrica: encarga.

La persona de la ventanilla no sabe de dónde salen los productos, y no le hace falta. Lo que sabe es que existe **un contrato de suministro** —«alguien me tiene que entregar el catálogo y buscarme un producto por su número»— y que, al empezar el turno, encuentra sobre su mesa a **un proveedor acreditado** que cumple ese contrato.

Quién es ese proveedor lo decide la administración de la oficina, no la persona de la ventanilla. Y ahí está toda la gracia: **se puede cambiar de proveedor sin tocar la ventanilla**.

El contrato es la interfaz. El proveedor acreditado es la clase anotada. La administración que decide es el contenedor.

Y el problema del laboratorio también es de oficina: **el día que hay dos proveedores acreditados para el mismo contrato, la administración se planta y no abre** hasta que alguien diga cuál es el titular.

3 Los pasos

3.1 Paso 1 · El dato y el contrato

Qué vamos a hacer

Escribir el objeto que se mueve —un producto— y el contrato de suministro, que es una interfaz sin ninguna implementación todavía.

Para entenderlo mejor

Redactar el contrato antes de buscar proveedor. Dice **qué hay que entregar**, no quién lo entrega ni de dónde lo saca.

El problema

Si la ventanilla habla directamente con una clase concreta —«llama a `ProductoRepositoryLista`»— queda casada con ella. Cambiar de origen de datos obliga a tocar la ventanilla, y probarla obliga a arrastrar el origen de datos de verdad.

La alternativa, y por qué no

Puedes escribir la clase concreta y usarla directamente. Con un solo origen de datos funciona y es más corto. Deja de funcionar el día que haya dos —el de verdad y el de pruebas—, que es exactamente lo que va a pasar en el paso 4 de este laboratorio.

Se pega

Archivo **nuevo** practica/src/main/java/cl/dgt/di/models/Producto.java:

```
package cl.dgt.di.models;

public record Producto(Long id, String nombre, int precio) {
}
```

Archivo **nuevo** practica/src/main/java/cl/dgt/di/repositories/ProductoRepository.java:

```
package cl.dgt.di.repositories;

import cl.dgt.di.models.Producto;

import java.util.List;
import java.util.Optional;

public interface ProductoRepository {

    List<Producto> todos();

    Optional<Producto> porId(Long id);
}
```

Es una interfaz y no tiene ninguna anotación. No es un componente: es el contrato.

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

La aplicación arranca normal. Todavía no hay nada que ver por HTTP: acabas de escribir un contrato que nadie firma aún.

Vas bien si...

Arranca sin errores. Si tu editor se queja de que la interfaz no se usa, tiene razón: todavía no.

Si te atascas

1 • **class, interface, enum, or record expected**

Se perdió una llave al copiar, o el archivo quedó a medias. Compara con el bloque.

2 • **duplicate class o el paquete no coincide.**

El archivo tiene que estar en la carpeta que dice su package: models/ para Producto, repositories/ para la interfaz.

3.2 Paso 2 · Una implementación, y una ventanilla que la usa

Qué vamos a hacer

Escribir un proveedor que cumple el contrato, y un controlador que lo pide **sin construirlo**.

Para entenderlo mejor

Acreditar al primer proveedor —colgarle el sello de «proveedor autorizado», que es la anotación— y abrir la ventanilla diciendo en el turno de mañana: «necesito un proveedor de productos».

El problema

Si el controlador escribe `new ProductoRepositoryLista()`, es él quien decide el proveedor. Y eso significa que para cambiarlo hay que abrir el controlador, y que para probarlo hay que aceptar el proveedor real.

La alternativa, y por qué no

- **new directo**: simple y honesto con una sola implementación; imposible de cambiar sin tocar el código que lo usa.
- **Una fábrica o un ServiceLocator**: el controlador pide «dame el repositorio» a un registro global. Mejor que el `new`, pero el controlador sigue teniendo que ir a buscarlo, y ahora depende además del registro.
- **Inyección por constructor**, que es lo que se usa aquí: el controlador **no busca nada**. Sus necesidades están en la firma del constructor, a la vista, y quien lo construye decide con qué. De regalo, la clase se puede construir con un `new` en un test pasándole lo que quieras — cosa que verás en el Lab 08.

Se pega

Archivo **nuevo** practica/src/main/java/cl/dgt/di/repositories/ProductoRepositoryLista.java:

```
package cl.dgt.di.repositories;

import cl.dgt.di.models.Producto;
import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
@Primary
public class ProductoRepositoryLista implements ProductoRepository {

    private static final List<Producto> DATOS = List.of(
        new Producto(1L, "Resma de papel carta", 4990),
```

```

        new Producto(2L, "Tóner negro", 68900),
        new Producto(3L, "Silla ergonómica", 129900),
        new Producto(4L, "Monitor 24 pulgadas", 149900));

@Override
public List<Producto> todos() {
    return DATOS;
}

@Override
public Optional<Producto> porId(Long id) {
    return DATOS.stream().filter(p -> p.id().equals(id)).findFirst();
}
}

```

Ese @Primary es del paso 5. Está aquí porque el archivo se pega una sola vez, pero hoy no hace nada: sólo hay un candidato. En el paso 4 vas a ver qué pasa sin él, y en el paso 5 se explica.

Archivo **nuevo** practica/src/main/java/cl/dgt/di/controllers/ProductoController.java:

```

package cl.dgt.di.controllers;

import cl.dgt.di.models.Producto;
import cl.dgt.di.repositories.ProductoRepository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/productos")
public class ProductoController {

    private final ProductoRepository repositorio;

    public ProductoController(ProductoRepository repositorio) {
        this.repositorio = repositorio;
    }

    @GetMapping
    public List<Producto> listar() {
        return repositorio.todos();
    }

    @GetMapping("/quien")
    public String quien() {
        return repositorio.getClass().getSimpleName();
    }
}

```

No hay un solo `new` en ese archivo. El constructor recibe lo que necesita, y punto.

Esta versión del controlador es provisional, y conviene saberlo desde ya: hoy pide directamente el `ProductoRepository` porque todavía no existe nada entre medias. En el paso 6 vas a reescribirlo para que pida un `ProductoService`, y ahí se explica por qué.

Se corre

```
curl localhost:8083/productos
```

Lo que vas a ver

```
[
  {"id": 1, "nombre": "Resma de papel carta", "precio": 4990},
  {"id": 2, "nombre": "Tóner negro", "precio": 68900},
  {"id": 3, "nombre": "Silla ergonómica", "precio": 129900},
  {"id": 4, "nombre": "Monitor 24 pulgadas", "precio": 149900}
]
```

Vas bien si...

`/productos` devuelve los cuatro productos, y `/productos/2` devuelve el tóner. Y en tu código no hay ni un `new`.

Si te atascas

1 · La aplicación no arranca y dice `required a bean of type ... that could not be found`.

Le falta el sello al proveedor: `@Repository` encima de la clase de implementación. Sin él, el contenedor no lo ve y el controlador se queda sin lo que pidió.

2 · 404 en `/productos`.

Falta `@RestController` o `@RequestMapping("/productos")`, o el archivo está fuera del paquete `cl.dgt.di`.

3.3 Paso 3 · La pregunta del laboratorio

Qué vamos a hacer

Añadir un endpoint que responda **quién te está atendiendo**, preguntándoselo al propio objeto que te inyectaron.

Para entenderlo mejor

Preguntarle al proveedor que tienes delante: «¿usted quién es?». Hasta ahora sabías que había uno; ahora vas a ver su nombre.

El problema

La inyección de dependencias se explica mil veces con diagramas y se entiende una vez que se ve el nombre de la clase real salir por una URL. Sin eso, «alguien te lo da» suena a magia.

Se pega

Este método va en el servicio del paso 6, pero puedes ponerlo hoy en el controlador para verlo funcionar antes:

```
public String quienMeAtiende() {  
    return repositorio.getClass().getSimpleName();  
}
```

Y el endpoint que lo expone:

```
@GetMapping("/quien")  
public String quien() {  
    return servicio.quienMeAtiende();  
}
```

Se corre

```
curl localhost:8083/productos/quien
```

Lo que vas a ver

```
ProductoRepositoryLista
```

Ese nombre no está escrito en ninguna parte de tu controlador. Lo puso el contenedor, y el objeto lo dice de sí mismo.

Vas bien si...

/productos/quien devuelve ProductoRepositoryLista, y puedes señalar en el código que ese texto no aparece escrito en ningún sitio.

Si te atascas

1 • /productos/quien da 404 y /productos/2 funciona.

Spring está tomando quien por un {id}. Si tu método de /{id} recibe un Long, la conversión falla. Pon el método /quien **antes** en la clase, o comprueba que la ruta esté escrita exactamente así.

3.4 Paso 4 · Dos candidatos: la aplicación deja de arrancar

Qué vamos a hacer

Añadir una **segunda** implementación del mismo contrato, y ver la aplicación negarse a arrancar. Este paso está pensado para fallar.

Para entenderlo mejor

Llega un segundo proveedor acreditado, con el mismo contrato y el mismo sello. La administración mira los dos papeles, ve que son igual de válidos, y **no abre la oficina**: no le toca a ella elegir a dedo.

El problema

Cuando hay dos candidatos igual de buenos, cualquier elección automática sería arbitraria. Y una elección arbitraria que funciona hoy es una bomba: mañana entra otra clase en el classpath, el orden cambia, y tu aplicación empieza a usar el proveedor de pruebas en producción **sin decir nada**.

La alternativa, y por qué no

Un contenedor podría elegir por orden alfabético, o por el primero que encuentre. Sería más cómodo hoy y un desastre dentro de seis meses, porque el fallo aparecería en producción y en silencio.

Fallar al arrancar es ruidoso, pasa en el despliegue, y lo arregla quien está mirando. Es exactamente el mismo criterio que verás en el Lab 13 con la clave de firma.

Se pega

Archivo **nuevo** practica/src/main/java/cl/dgt/di/repositories/ProductoRepositoryFalso.java:

```
package cl.dgt.di.repositories;

import cl.dgt.di.models.Producto;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public class ProductoRepositoryFalso implements ProductoRepository {

    private static final List<Producto> DATOS = List.of(
        new Producto(1L, "PRODUCTO DE PRUEBA UNO", 1),
        new Producto(2L, "PRODUCTO DE PRUEBA DOS", 2));

    @Override
    public List<Producto> todos() {
        return DATOS;
    }

    @Override
    public Optional<Producto> porId(Long id) {
        return DATOS.stream().filter(p -> p.id().equals(id)).findFirst();
    }
}
```

Y ahora, **para ver el fallo**, quita temporalmente el @Primary de ProductoRepositoryLista —la anotación y su import—.

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

No arranca, y el mensaje es de los buenos:

```
*****  
APPLICATION FAILED TO START  
*****
```

Description:

```
Parameter 0 of constructor in cl.dgt.di.services.ProductoService required a single  
↪ bean,  
but 2 were found:  
- productoRepositoryFalso: defined in file [.../ProductoRepositoryFalso.class]  
- productoRepositoryLista: defined in file [.../ProductoRepositoryLista.class]
```

Léelo entero. Dice **quién** lo pedía, **cuántos** encontró y **cuáles son**. No hay que adivinar nada.

Vas bien si...

La aplicación **no arranca**, y el mensaje nombra los dos candidatos. Si arrancó, todavía tienes el `@Primary` puesto: quítalo y vuelve a probar — este paso hay que verlo fallar.

Si te atascas

1 · Arranca igual.

No quitaste el `@Primary`, o quitaste la anotación y dejaste el `import` (eso no basta: lo que decide es la anotación). O el archivo nuevo no está en el paquete `cl.dgt.di.repositories`.

2 · Dice `could not be found` en vez de `2 were found`.

Es el problema contrario: ninguno de los dos tiene `@Repository`.

3.5 Paso 5 · Las dos formas de resolverlo

Qué vamos a hacer

Elegir proveedor, de las dos maneras que existen, y entender cuándo se usa cada una.

Para entenderlo mejor

Dos formas de resolver el atasco de la oficina:

- **Nombrar un titular:** «el proveedor de la lista es el titular; salvo que alguien pida otra cosa, se le llama a él». Eso es `@Primary`.
- **Pedir uno por su nombre:** la ventanilla dice «yo quiero específicamente al proveedor de pruebas». Eso es `@Qualifier`.

El problema

Hay que desempatar, pero **dónde** se pone el desempate cambia mucho las cosas. Ponerlo junto al candidato dice «éste es el normal». Ponerlo junto a quien pide dice «yo, y sólo yo, quiero éste».

Cuál se elige aquí, y por qué

@Primary, y va sobre ProductoRepositoryLista. La razón es que **hay un caso normal y un caso excepcional**: la lista es el proveedor de siempre y el falso es el de pruebas. Con @Primary, todo el que pida un repositorio recibe el normal sin tener que decir nada, y sólo quien necesite el otro tiene que pedirlo expresamente.

Con @Qualifier en todas partes pasaría lo contrario: cada sitio que pide tendría que nombrar al proveedor, y añadir una implementación nueva obligaría a revisarlos todos.

@Qualifier es la respuesta cuando no hay un caso normal — cuando hay tres estrategias de cálculo y cada quien usa la suya a conciencia.

Se pega

Vuelve a poner en ProductoRepositoryLista.java lo que quitaste en el paso 4:

```
import org.springframework.context.annotation.Primary;
```

y la anotación, junto a @Repository:

```
@Primary
```

Se corre

```
./mvnw spring-boot:run  
curl localhost:8083/productos/quien
```

Lo que vas a ver

ProductoRepositoryLista

Arranca otra vez, y el endpoint del paso 3 te dice cuál de los dos ganó. **Los dos candidatos siguen existiendo**: lo que cambió es que ahora hay un titular.

Vas bien si...

La aplicación arranca con las dos implementaciones presentes, y /productos/quien responde ProductoRepositoryLista.

Si te atascas

1 · Sigue sin arrancar.

Pusiste el @Primary pero falta su import, o lo pusiste en la interfaz en vez de en la clase. Va sobre la **implementación** que quieres que gane.

2 · Responde ProductoRepositoryFalso.

Pusiste el @Primary en la clase equivocada. Es un buen error para tener a la vista: así de fácil sería enviar a producción los datos de prueba, y así de visible lo hace este endpoint.

3.6 Paso 6 · La capa del medio

Qué vamos a hacer

Meter un servicio entre el controlador y el repositorio.

Para entenderlo mejor

Entre la ventanilla y el proveedor hay un **jefe de compras**. La ventanilla atiende al público; el proveedor entrega mercancía; el jefe de compras es quien sabe las reglas del negocio — qué se puede pedir, cuánto, con qué condiciones.

El problema

Un controlador que llama directo al repositorio funciona mientras no haya reglas. En cuanto aparece la primera —«no mostrar los productos dados de baja», «aplicar el descuento vigente»—, esa regla tiene que vivir en algún sitio. Si vive en el controlador, se queda atada a HTTP: el día que la misma regla haga falta desde una tarea programada o desde otro endpoint, hay que copiarla.

La alternativa, y por qué no

Para este laboratorio, el servicio **no hace nada**: llama al repositorio y devuelve. Se puede argumentar que sobra, y con este código sobra.

Se pone igual por una razón concreta: es la capa donde van a vivir las transacciones del Lab 07 y las reglas de negocio de todos los labs siguientes. Añadirla ahora cuesta quince líneas; añadirla cuando ya hay diez controladores llamando al repositorio cuesta una tarde.

Se pega

Archivo **nuevo** practica/src/main/java/cl/dgt/di/services/ProductoService.java:

```
package cl.dgt.di.services;

import cl.dgt.di.models.Producto;
import cl.dgt.di.repositories.ProductoRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class ProductoService {

    private final ProductoRepository repositorio;

    public ProductoService(ProductoRepository repositorio) {
        this.repositorio = repositorio;
    }

    public List<Producto> catalogo() {
        return repositorio.todos();
    }
}
```

```

    public Optional<Producto> porId(Long id) {
        return repositorio.porId(id);
    }

    public String quienMeAtiende() {
        return repositorio.getClass().getSimpleName();
    }
}

```

Y en el controlador, **cambia el constructor**: donde pedía un ProductoRepository, ahora pide un ProductoService. Así queda el archivo entero:

```

package cl.dgt.di.controllers;

import cl.dgt.di.models.Producto;
import cl.dgt.di.services.ProductoService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/productos")
public class ProductoController {

    private final ProductoService servicio;

    public ProductoController(ProductoService servicio) {
        this.servicio = servicio;
    }

    @GetMapping
    public List<Producto> listar() {
        return servicio.catalogo();
    }

    @GetMapping("/quien")
    public String quien() {
        return servicio.quienMeAtiende();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Producto> porId(@PathVariable Long id) {
        return servicio.porId(id)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}

```

Fíjate en `@Service`: es lo mismo que `@Repository` en lo que hace —las dos son componentes—, pero **dice qué es la clase**. El nombre es para quien lee.

Se corre

```
curl localhost:8083/productos/quien
curl localhost:8083/productos
```

Lo que vas a ver

Exactamente lo mismo que antes. **Y eso es el resultado**: metiste una capa entera por en medio y quien llama desde fuera no se enteró.

Vas bien si...

Los tres endpoints siguen funcionando igual que en el paso 5, y ahora el controlador no conoce al repositorio: sólo conoce al servicio.

Si te atascas

1 · required a bean of type 'cl.dgt.di.services.ProductoService' that could not be found

Falta `@Service` encima de la clase, o el archivo no está bajo `cl.dgt.di`.

2 · El controlador no compila.

Quedó el campo viejo del repositorio junto al nuevo del servicio. Tiene que quedar sólo el servicio; compara con el bloque del archivo entero.

4 Lo que aprendiste

1 · Tú declaras lo que necesitas; otro decide quién te lo da.

Eso es Spring, y hoy lo comprobaste: en tu controlador no hay un solo `new`, y aun así tiene lo que pidió. Las necesidades van en el constructor, que es donde se pueden leer.

2 · El contrato es una interfaz; el proveedor es una clase anotada.

Programar contra la interfaz es lo que permitió tener dos implementaciones a la vez y cambiar de una a otra sin tocar ni el controlador ni el servicio.

3 · Con dos candidatos, el contenedor se planta — y eso es una virtud.

Podría haber elegido uno al azar y funcionar. Habría sido peor: el fallo aparecería en producción, en silencio, el día que cambiara el orden del classpath. Fallar al arrancar es la mejor hora para fallar.

4 · `@Primary` y `@Qualifier` responden a preguntas distintas.

`@Primary` dice «éste es el normal» y va junto al candidato. `@Qualifier` dice «yo quiero éste» y va junto a quien pide. Si hay un caso normal, `@Primary`; si no lo hay, `@Qualifier`.

5 Para profundizar

- **Pon @Primary en el falso** y mira /productos/quien. Después imagina ese despiste en un despliegue de verdad.
- **Quita el @Primary y usa @Qualifier("productoRepositoryFalso")** en el constructor del servicio. Comprueba que el nombre por defecto de un bean es el de la clase con la primera letra en minúscula.
- **Añade un tercer método al contrato** y no lo implementes en una de las dos clases. ¿Cuándo te enteras: al compilar o al arrancar?
- **Pon un System.out.println en el constructor** de las dos implementaciones y arranca. ¿Se construyen las dos, o sólo la que gana?

6 Antes de cerrar

Para la aplicación con Ctrl+C — si no, el puerto 8083 se queda ocupado.

```
./mvnw clean
```

Lo que te llevas:

Una clase declara lo que necesita en su constructor y no construye nada. El contenedor busca un candidato que cumpla el contrato y se lo entrega. Si hay dos y ninguno es el titular, no arranca.

Lo que queda pendiente, y abre el Lab 03: hoy, cuando pediste un producto que no existía, recibiste un 404 con el cuerpo vacío. Vacío no le sirve a quien llama: no dice qué pasó, ni cuál de los datos estaba mal. En el Lab 03 los errores empiezan a explicarse.